

① Star Pattern:

→ Square :-

Logic:

→ will use nested loops

→ inner loop prints the "*" ($j < 5$)

→ outer loop for rows ($i < 5$), inner loop for columns.

Code:

```
for (int i = 0; i < 5; i++) {  
    for (int j = 0; j < 5; j++) {  
        cout << " * ";  
    }  
    cout << endl;  
}
```

→ Hollow Square :-

Logic:

→ use nested loops

→ outer loop for rows ($i < 5$)

→ inner loop for columns ($j < 5$)

→ condition of inner loop:

Print "*" if ($i == 0 || i == 4 || j == 0 || j == 4$)

else print " ";

This is the matrix logic.

```
#include <iostream>
using namespace std;
int main()
{
    for (int i = 0; i < 5; i++)
    {
        for (int j = 0; j < 5; j++)
        {
            cout << '*';
        }
        cout<<endl;
    }

    return 0;
}
```

● PS C:\Users\hp\Desktop\3rd-Yr-Week-1-Module-1>

```
*****
```

```
*****
```

```
*****
```

```
*****
```

```
*****
```

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 0; i <5; i++) {
        for (int j = 0; j < 5; j++) {
            if (i == 0 || i == 4 || j == 0 || j == 4)
                cout << "**";
            else
                cout << " ";
        }
        cout << endl;
    }

    return 0;
}
```

● PS C:\Users\hp\Desktop\3rd-Yr-Week-1-Module-1>

```
*****
```

```
*      *
```

```
*      *
```

```
*      *
```

```
*****
```

code:

```
for (int i = 0; i < 5; i++) {  
    for (int j = 0; j < 5; j++) {  
        if (i == 0 || i == 4 || j == 0 || j == 4) {  
            cout << "*";  
        }  
        else {  
            cout << " ";  
        }  
        cout << endl;  
    }  
}
```

→ Triangle:-

logic:

- use nested loops
- outer for Rows ($i < 5$)
- inner for columns ($j \leq i$)
- every iteration of inner loop prints "*" &

code:

```
for (int i = 0; i < 5; i++) {  
    for (int j = 0; j <= i; j++) {  
        cout << "*";  
    }  
    cout << endl;  
}
```

```
#include <iostream>
using namespace std;
int main()
{
    for (int i = 0; i < 5; i++)
    {
        for (int j = 0; j <= i; j++)
        {
            cout << "*";
        }
        cout << endl;
    }

    return 0;
}
```

● PS C:\Users\hp\Desktop\3rd-Yr-Week-1-1

```
*
**
***
****
*****
```

→ Inverted Triangle:-

Logic:

→ use nested loops

→ 1 outer loop ($i \geq -1$)

→ 2 inner loops ($s \leq 5-i$)

($j \leq i$)
→ ~~first~~ first inner loop for spaces

→ Second inner loop for element "*" ;

code:

```
for (int i = 5; i >= 1; i--) {
```

```
    for (int s = 0; s < 5 - i; s++) {
```

```
        cout << " "; }
```

```
    for (int j = 1; j <= i; j++) {
```

```
        cout << "*" ; }
```

```
    cout << endl; }
```

→ Pyramid:-

Logic:

→ use nested loops

→ 1 outer loop for rows

→ 3 inner loops

→ 1st inner loop for left spaces ($s < n-i-1$)

→ 2nd inner loop for stars ($j \leq 2*i+1$)

→ 3rd inner loop for Right spaces ($s < n-i-1$)

```

#include <iostream>
using namespace std;
int main()
{
    for (int i = 5; i >= 1; i--)
    {
        for(int s=0;s<5-i;s++){
            cout<<" ";
        }

        for (int j = 1; j <=i; j++)
        {
            cout << "* ";
        }
        cout << endl;
    }
    return 0;
}

```

```

● PS C:\Users\hp\
module\3rd-Yr-W
* * * * *
    * * * *
        * * *
            * *
                *

```

```

#include <iostream>
using namespace std;
int main()
{
    int n;
    cout << "Enter n: ";
    cin >> n;
    for (int i = 0; i < n ; i++)
    {
        for (int s = 0; s < n -i-1; s++)
        {
            cout << " ";
        }

        for (int j = 1; j <= 2 * i + 1; j++)
        {
            cout << "*";
        }
        for (int s = 0; s < n -i- 1; s++)
        {
            cout << " ";
        }
        cout << endl ;
    }
    return 0;
}

```

```

● PS C:\Users\hp\Des
e\3rd-Yr-Week-1-Mo
Enter n: 5
    *
    ***
    *****
    *********
    *********

```

code:

```
for (int i = 0; i < n; i++) {
```

```
    for (int s = 0; s < n - i - 1; s++) {
```

```
        cout << " ";
```

```
    }  
    for (int j = 1; j <= 2 * i + 1; j++) {
```

```
        cout << "*";
```

```
    }  
    for (int s = 0; s < n - i - 1; s++) {
```

```
        cout << " ";
```

```
    }  
    cout << endl;
```

→ Diamond :-

Logic :

→ Using two sets of nested loops, one for ~~upper~~ upper half other for lower half.

→ Each set of nested loop has one outer loop and two inner loop.

→ 1st inner loop used to print space ($i < 5 - i - 1$)

→ 2nd inner loop to print elements ($j < 2 * i + 1$)

```

#include <iostream>
using namespace std;

int main() {

    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 5 - i - 1; j++)
            cout << " ";

        for (int j = 0; j < 2 * i + 1; j++)
            cout << "*";

        cout << endl;
    }

    for (int i = 3; i >= 0; i--) {
        for (int j = 0; j < 5 - i - 1; j++)
            cout << " ";

        for (int j = 0; j < 2 * i + 1; j++)
            cout << "*";

        cout << endl;
    }

    return 0;
}

```

```

PS C:\Users\h...
module\3rd-Yr...

      *
    ***
  *****
*****
*****
*****
*****
  *****
    ***
      *

```

code:

```
for (int i = 0; i < 5; i++) {  
    for (int j = 0; j < 5 - i - 1; j++) {  
        cout << " "; }  
}
```

```
for (int j = 0; j < 2 * i + 1; j++) {  
    cout << "*"; }  
}
```

```
cout << endl; }
```

```
for (int i = 3; i >= 0; i--) {
```

```
    for (int j = 0; j < 5 - i - 1; j++) {  
        cout << " "; }
```

```
for  
for (int j = 0; j < 2 * i + 1; j++) {  
    cout << "*"; }
```

```
cout << endl; }
```

→ Half Diamond :-

Logic:

- using two sets of nested loops, having one outer and one inner loop.
- Inner loop used to print "*" ($j = i$)
- Nested loop sets used for upper and lower half

```

#include <iostream>
using namespace std;

int main() {

    for (int i = 0; i < 5; i++) {
        for (int j = 0; j <= i; j++) {
            cout << "*";
        }
        cout << endl;
    }

    for (int i = 3; i >= 0; i--) {
        for (int j = 0; j <= i; j++) {
            cout << "*";
        }
        cout << endl;
    }

    return 0;
}

```

```

● PS C:\Users\hp\OneDrive\Desktop> g++ module\3rd-Yr-Week-1\3rd-Yr-Week-1.cpp
*
**
***
****
*****
****
***
**
*

```

code:

```
for (int i = 0; i < 5; i++) {  
    for (int j = 0; j <= i; j++) {  
        cout << " * ";  
    }  
    cout << endl;  
}
```

```
for (int i = 3; i >= 0; i--) {  
    for (int j = 0; j <= i; j++) {  
        cout << " * ";  
    }  
    cout << endl;  
}
```

→ Half Diamond Pattern :-

Logic :

→ using nested loops, one outer loop and two inner loops

→ 1st inner loop used to print spaces ($j < i$)

→ 2nd inner loop used to print "*" ($j < 5 - i$)

code:

```
for (int i = 0; i < 5; i++) {  
    for (int j = 0; j < i; j++) {  
        cout << " ";  
    }  
    for (int j = 0; j < 5 - i; j++) {  
        cout << " * ";  
    }  
    cout << endl;  
}
```

```

#include <iostream>
using namespace std;

int main() {

    for (int i = 0; i < 5; i++) {

        for (int j = 0; j < i; j++) {
            cout << " ";
        }

        for (int j = 0; j < 5 - i; j++) {
            cout << "* ";
        }

        cout << endl;
    }

    return 0;
}

```

```

● PS C:\Users\hp\Documents\module\3rd-Yr-
    * * * * *
      * * * *
        * * *
          * *
            *

```

② Number Star Patterns

→ Square :-

Logic:

→ use nested loops

→ outer loop for rows

→ inner loop for ~~column~~ columns.

→ Print "*" for every iteration of inner loop.

Code:

```
for(int i=0; i<5; i++){  
    for(int j=0; j<5; j++){  
        cout << "* ";  
    }  
    cout << endl;  
}
```

→ ~~Hollow Square~~ Incrementing:

Logic:

→ use nested loops

→ Declare num = 1 inside first loop

→ inner ~~loop~~ loop for columns.

→ Print num ~~for every iteration~~ for every iteration of inner loop then num++.

```
#include<iostream>
using namespace std;
int main(){
    for(int i=0;i<5;i++){
        for(int j=0;j<5;j++){
            cout<<"1";
        }cout<<endl;
    }
    return 0;
}
```

● PS C:\Users\h
module\3rd-Yr
11111
11111
11111
11111
11111

```
#include<iostream>
using namespace std;
int main(){
    for(int i=0;i<5;i++){
        int num=1;
        for(int j=0;j<5;j++){
            cout<<num;
            num++;
        }cout<<endl;
    }
    return 0;
}
```

● PS C:\Users\h
module\3rd-Yr
12345
12345
12345
12345
12345

code:

```
for(int i=0; i<5; i++){  
    int num=1;  
    for(int j=0; j<5; j++){  
        cout<<num;  
        num++;  
    }  
    cout<<endl;  
}
```

Right
→ Triangle: -
Logic:

- use nested loops
- declare ~~at~~ num=1 inside first loop
- condition for inner loop ($j \leq i$),
to limit no of elements in a row
- inner loop prints num in every iteration.

code:

```
for(int i=0; i<5; i++){  
    int num=1;  
    for(int j=0; j<=i; j++){  
        cout<<num;  
        num++;  
    }  
    cout<<endl;  
}
```

```
#include<iostream>
using namespace std;
int main(){

    for(int i=0;i<5;i++){
        int num=1;
        for(int j=0;j<=i;j++){
            cout<<num;
            num++;
        }cout<<endl;
    }
    return 0;
}
```

```
● PS C:\Users\hp\
  module\3rd-Yr-w
    1
    12
    123
    1234
    12345
```

→ Inverted ^{Diamond} ~~triangle~~:

Logic:

→ we nested loops

→ 1 outer and 2 inner loops

→ First inner loop to print blank spaces

→ Second inner loop to print elements + spaces

code:

```
for (int i=0; i<5; i++) {
```

```
    for (int j=0; j<i; j++) {
```

```
        cout << " "; }
```

```
    for (int j=0; j<5-i; j++) {
```

```
        cout << i+3 << " "; }
```

```
    cout << endl; }
```

→ Sandwich Pattern:

Logic:-

→ we will use two sets of nested loops

→ Both contain 1 outer and 2 inner loop

→ 1st inner loop is used to print spaces

→ 2nd inner loop is used to print elements
($j < 5 - 2*i$)

```

#include <iostream>
using namespace std;

int main() {

    for (int i = 0; i < 5; i++) {

        for (int j = 0; j < i; j++) {
            cout << " ";
        }

        for (int j = 0; j < 5 - i; j++) {
            cout << i + 3 << " ";
        }

        cout << endl;
    }

    return 0;
}

```

```

● PS C:\Users\hp\Documents\module\3rd-Yr-W
3 3 3 3 3
  4 4 4 4
    5 5 5
      6 6
        7

```

→ First set of nested loop is used to print the upper part, second set is used for the lower part.

$(i < 3) (i >= 0)$

1st

2nd

code:

```
for (int i = 0; i < 3; i++) {
```

```
    for (int j = 0; j < i; j++) {
```

```
        cout << " ";
```

```
    for (int j = 0; j < 5 - 2 * i; j++) {
```

```
        cout << i + 3 << " ";
```

```
    cout << endl;
```

```
for (int i = 1; i >= 0; i--) {
```

```
    for (int j = 0; j < i; j++) {
```

```
        cout << " ";
```

```
    for (int j = 0; j < 5 - 2 * i; j++) {
```

```
        cout << i + 3 << " ";
```

```
    cout << endl;
```

```

#include <iostream>
using namespace std;

int main() {

    for (int i = 0; i < 3; i++) {

        for (int j = 0; j < i; j++)
            cout << " ";

        for (int j = 0; j < 5 - 2 * i; j++)
            cout << i + 3 << " ";

        cout << endl;
    }

    for (int i = 1; i >= 0; i--) {

        for (int j = 0; j < i; j++)
            cout << " ";

        for (int j = 0; j < 5 - 2 * i; j++)
            cout << i + 3 << " ";

        cout << endl;
    }

    return 0;
}

```

- PS C:\Users\hp\Desktop\module\3rd-Yr-Week

3 3 3 3 3

4 4 4

5

4 4 4

3 3 3 3 3

→ Incrementing diamond pattern

Logic:

- We use two set of nested loops
- Each contain one outer and one inner loop.
- First set of nested loop is used for the ~~upper~~ upper part.
- inner loop prints the elements ($j \leq i$)
- Second set of nested loop is used for the lower part.

Code:

```
for (int i = 0; i < 5; i++) {  
    for (int j = 0; j <= i; j++) {  
        cout << i + j << " ";  
    }  
    cout << endl;
```

```
}  
for (int i = 3; i >= 0; i--) {  
    for (int j = 0; j <= i; j++) {  
        cout << i + j << " ";  
    }  
    cout << endl;
```

```

#include <iostream>
using namespace std;

int main() {

    for (int i = 0; i < 5; i++) {

        for (int j = 0; j <= i; j++)
            cout << i + 3 << " ";

        cout << endl;
    }

    for (int i = 3; i >= 0; i--) {

        for (int j = 0; j <= i; j++)
            cout << i + 3 << " ";

        cout << endl;
    }

    return 0;
}

```

```

● PS C:\Users\hp\Desktop\module\3rd-Yr-Week
3
4 4
5 5 5
6 6 6 6
7 7 7 7 7
6 6 6 6
5 5 5
4 4
3

```